

Assertion and Question Developer for Agentic Questionnaire Development

Lanre Oriowo
Ludwig-Maximilian-University
Munich, Germany
L.Oriowo@campus.lmu.de

Rudraksha Samdhani
Ludwig-Maximilian-University
Munich, Germany
Rudraksha.Samdhani@campus.lmu.de

Abstract

Designing high-quality survey items is expert-intensive and slow, a bottleneck for empirical computational social science. We ask how far prompt engineering alone, without fine-tuning, can drive an open-weight large language model to generate items adhering to a formal survey-design framework. We implement the three-step design method, which maps an indicator to a basic concept and semantic structure, then to an assertion, then to a respondent-facing question, as a two-agent pipeline on a 9-billion-parameter model with constrained JSON decoding, evaluated against a hand-built gold set of 113 items spanning 22 basic concepts. Across seven controlled runs, concept accuracy rises from 65.5 to 85.0 percent and semantic-structure accuracy from 51.3 to 79.6 percent. Wilson intervals and paired McNemar tests show that only two of six transitions are significant, and that much of the largest early gain came from a change in the scoring normalizer, not the model. Two controls qualify the evaluation: a two-by-two generator-by-judge design finds no self-preference bias, but exposes the smaller judge as a degenerate instrument, placing 1 percent of items mid-scale against the larger judge’s 38 percent; and a 32-billion-parameter generator scores lower on the identical prompt, indicating that framework adherence is not primarily limited by model scale.

Index Terms

survey methodology, large language models, prompt engineering, question generation, structured decoding, LLM-as-a-judge, computational social science

Contents

I	Introduction	3
II	Background and Related Work	4
II-A	Survey Design Frameworks	4
II-B	LLMs for Question and Item Generation	4
II-C	LLM-as-a-Judge and Self-Grading Bias	4
II-D	Constrained and Structured Decoding	4
II-E	Open-Weight Model Serving	5
II-F	Positioning	5
III	Framework and Task Formalization	5
III-A	Concepts, Structures, and Notation	5
III-B	Task Definition	6

IV	Method: The Two-Agent Pipeline	7
IV-A	Dynamic Framework Injection	7
IV-B	Assertion Developer	8
IV-C	Question Developer	8
IV-D	Structured Decoding and Robustness	8
IV-E	Model and Serving	8
V	Experimental Setup	9
V-A	Gold Set	9
V-B	Isolated Evaluation Protocol	9
V-C	Normalization	9
V-D	Metrics	10
V-E	Judge Models	10
V-F	Reproducibility	10
VI	Prompt-Engineering Iterations	11
VII	Results	12
VII-A	Overall Progression	12
VII-B	Per-Concept and Per-Structure Behavior	13
VII-C	Question Stage	14
VII-D	Semantic (Judge) Results	15
VIII	Error Analysis and Discussion	17
VIII-A	Concept Confusions	17
VIII-B	Structure Confusions and the Concept–Structure Coupling	17
VIII-C	A Prompt-Engineering Trade-off: The Norms Regression	18
VIII-D	Self- versus External-Judge Bias	18
VIII-E	Exact Match Understates Question Quality	18
VIII-F	Prompt Engineering versus Fine-Tuning	19
IX	Limitations	19
X	Conclusion and Future Work	20
	Appendix A: Notation Key	20
	Appendix B: Agent Prompts	21
	Appendix C: Complete Per-Concept Results	22
	Appendix D: Confusion Matrices	23
	Appendix E: Judge Rubric and Hardest Items	26
	Appendix F: Reproduction Details	26
	References	26

I. Introduction

Surveys are a primary instrument of empirical social science, yet the quality of the data they produce is bounded by the quality of their individual items. Designing items that are valid, unambiguous, and comparable across studies is a specialized craft, requiring domain expertise, familiarity with measurement theory, and repeated expert review [1]. It is therefore slow, costly, and difficult to reproduce, which constrains large-scale and cross-national research in computational social science, where hundreds of indicators may need to be turned into well-formed questions.

A substantial body of survey methodology argues that item construction should not be ad hoc but should follow an explicit procedure that separates the concept being measured from its linguistic realization. In particular, the three-step design method decomposes item construction into (i) identifying the basic concept an indicator refers to together with its semantic structure, (ii) formulating a declarative assertion that expresses that concept in the prescribed structure, and (iii) transforming the assertion into a respondent-facing request for an answer, that is, a survey question with response options [2]. This framework grounds item design in a formal taxonomy of concepts and structures, which makes the output theory-consistent and, importantly for our purposes, checkable against a reference standard.

Large language models (LLMs) are natural candidates to automate this procedure: they are fluent, follow instructions, and can be steered with a small number of worked examples [3], [4]. However, applied naively they generate free-form text that ignores measurement frameworks, invents categories, and produces output that is hard to validate at scale. Two ingredients are needed to make LLM-based item generation useful for methodologists: the generation must be grounded in the concept-and-structure taxonomy, and it must be evaluable against that taxonomy rather than judged only by surface fluency.

In this paper we ask a focused question: how far can prompt engineering alone, without any model fine-tuning, drive an open-weight LLM to generate framework-faithful survey items? We implement the three-step method as a two-agent pipeline, an Assertion Developer and a Question Developer, built on a 9-billion-parameter model served with constrained JSON decoding, and evaluate it against a hand-built gold set of 113 items covering all 22 basic concepts under a dual protocol that separates framework adherence, exact match against the gold labels, from semantic fidelity, measured with an LLM judge [5].

The contributions of this report are:

- A reproducible two-agent pipeline that implements a formal survey-design framework, using dynamic taxonomy injection and constrained decoding to guarantee well-formed, in-taxonomy outputs.
- A dual evaluation protocol that distinguishes framework adherence from semantic fidelity, together with a two-by-two generator-by-judge control that separates self-preference from judge strictness. The control finds no self-preference bias, and instead shows that a small self-judge fails to use its own rating scale, which we argue is the more consequential finding for LLM-as-judge practice.
- An empirical baseline showing that seven controlled prompt-engineering runs raise concept accuracy from 65.5% to 85.0% and structure accuracy from 51.3% to 79.6% with no fine-tuning, reported with Wilson intervals and paired significance tests, and accompanied by a per-concept error analysis that isolates the remaining failure modes.

II. Background and Related Work

A. Survey Design Frameworks

The measurement quality of a survey item is not an intrinsic property of its wording but of how faithfully that wording realizes an intended concept [1]. To make this relationship explicit, the three-step design method distinguishes the basic concept an item measures from the semantic structure used to express it and from the final request for an answer presented to the respondent [2]. Concepts are organized into a taxonomy: subjective concepts capture internal states such as evaluations, feelings, preferences, norms, and values, whereas objective concepts capture verifiable facts such as behaviors, events, demographics, places, and quantities. Each concept admits only a small set of licensed semantic structures, written as compact codes (for example, a subject linked to an evaluative adjective, or a subject with a future deed) that constrain how the assertion may be phrased. Because the framework prescribes which concept and structure a given indicator should map to, it also defines an objective reference against which an automated system can be scored, a property we exploit in our evaluation.

B. LLMs for Question and Item Generation

Automatic question generation has a long history in education and information retrieval, where the goal is typically to produce comprehension or assessment questions from a source text [6]. Large language models have since become general-purpose text generators that follow natural-language instructions and adapt from a handful of in-context examples [3], [4], and a growing literature examines their use across computational social science tasks [7]. Applied to survey design, however, unconstrained generation tends to ignore measurement frameworks: models paraphrase indicators fluently but do not commit to a concept taxonomy or a licensed structure, which makes their output difficult to validate. Our work differs from generic question generation in that the target is not merely a fluent question but a framework-conformant chain of concept, structure, assertion, and question that can be checked against an expert taxonomy.

C. LLM-as-a-Judge and Self-Grading Bias

Because reference-based metrics such as exact match penalize valid paraphrases, recent evaluation practice uses a strong LLM as a judge that rates outputs on a rubric [5]. Judge models correlate well with human preferences but are known to exhibit biases, including position and verbosity effects and, most relevant here, a self-preference bias in which a model rates its own generations more favorably than an independent evaluator would [8], [9]. We therefore report judge scores from both the generating model itself and a larger, independent model, and we quantify the gap between them rather than relying on self-evaluation alone.

D. Constrained and Structured Decoding

Reliably extracting structured predictions from an LLM requires that its output conform to a fixed schema. Guided or constrained decoding restricts generation to tokens permitted by a grammar or JSON schema, which guarantees parseable output and can additionally restrict a field to an enumerated set of allowed values [10]. We use schema-guided JSON decoding at both pipeline stages and constrain the predicted concept to the enumerated taxonomy, which prevents the model from inventing labels and makes taxonomy-level scoring unambiguous.

E. Open-Weight Model Serving

Our experiments use open-weight models served locally, which avoids sending data to third parties and makes runs reproducible. We rely on a high-throughput inference server that uses paged attention for efficient memory management [11] to host an open-weight instruction-tuned model family [12], both for generation and, in a larger configuration, for the external judge.

F. Positioning

A small but directly adjacent literature has appeared alongside this work. Adhikari et al. [13] build an LLM pipeline that generates questionnaires, pretests them with a target audience, and adapts standardized instruments across contexts, evaluating with 238 participants in the US and 118 in South Africa; their participants rated LLM-generated text clearer and LLM-adapted questions less biased than the traditional counterparts. That work validates against human respondents, which our evaluation does not, whereas ours scores adherence to an explicit taxonomy, which theirs does not. Fuchs et al. [14] compare prompting strategies for survey-question generation across models and report chain-of-thought prompting as strongest; this is the closest existing point of comparison to our Runs 1 through 7, and we note that our own progression is a study of prompt content rather than of reasoning format, so the two are complementary rather than competing.

A further external instrument is available and we do not use it. The Survey Quality Predictor [15], developed in the same measurement tradition as the framework we implement [2], predicts the reliability and validity of a question from its formal and linguistic characteristics. Being neither an LLM nor reference-based, it shares the failure modes of neither of our evaluation axes, and would supply the methodologically-native quality metric that Section IX identifies as missing; we regard applying it as the single most valuable extension of our evaluation.

Relative to this work, our contribution is the combination of three elements: generation grounded in an explicit survey-design taxonomy through constrained decoding, a dual evaluation separating framework adherence from semantic fidelity, and a controlled study of how far prompt engineering alone can go before fine-tuning is warranted. The individual ingredients are not new; the combination, and in particular the taxonomy-grounded adherence metric, is what we have not found applied to automated survey item construction elsewhere.

III. Framework and Task Formalization

This section formalizes the survey-design framework we automate and states the two prediction tasks that our pipeline must solve.

A. Concepts, Structures, and Notation

An indicator i is a short natural-language phrase describing what an item should measure, for example “overall satisfaction with product” or “country of residence.” The framework maps each indicator to exactly one basic concept drawn from a fixed taxonomy $\mathcal{C}$ with $|\mathcal{C}| = 22$. The taxonomy is partitioned into 14 subjective concepts, which describe internal states (evaluations, feelings, preferences, norms, values, and so on), and 8 objective concepts, which describe verifiable facts (behaviors, events, demographics, places, quantities, and so on).

Each concept licenses a small set of semantic structures. A semantic structure specifies the grammatical skeleton of the assertion and is written as a compact structure code built from a shared notation. Three base templates underlie all codes, as summarized in

TABLE I: The Three Base Semantic-Structure Templates

#	Template	Example
1	subject + link verb + scale/classifier	“Someone is bad”
2	subject + predictor + object	“New laws changed X”
3	subject + predictor (no object)	“The position has changed”

TABLE II: Basic Concept Taxonomy and Licensed Structure Codes

Basic concept	Type	Licensed structure code(s)
Evaluation	subj.	xIe
Importance	subj.	xIi
Values	subj.	vIi, xIi
Feelings	subj.	xIf, xFy
Cognitive judgement	subj.	xIc
Causal relationship	subj.	$xIca, xCy$
Similarity relationship	subj.	xIs, xSy
Preference	subj.	$xIpr, xPRy$
Norms	subj.	$o(H+I)y, o(H+I)$
Policies	subj.	$g(H+I)y, g(H+I)$
Rights	subj.	$xIri, xHRy$
Action tendencies	subj.	xFD
Expectations of future events	subj.	xFD
Evaluative belief	subj.	$xP(e)y, xP(e)$
Behaviour	obj.	rDy, rD
Events	obj.	xDy, xD
Demographics	obj.	xId
Knowledge	obj.	$xIsc, xPy, xP$
Time	obj.	$xDti$
Place	obj.	$xDpl$
Quantities	obj.	$xDqu$
Procedures	obj.	$xDpl, pro^1$

Table I: a subject linked to a scale or classifier, a subject with a predictor acting on an object, and a subject with a predictor but no object. Codes specialize these templates with symbols denoting the subject type (for example x for the respondent or topic, r for the respondent as actor, o for a generic subject, g for a government), the linking element (I for a link verb, P for a predictor), and the object or complement (for example e for an evaluative adjective, i for an importance adjective, F for a feeling, D for a deed, FD for a future deed). Thus xIe denotes “respondent is [evaluative adjective],” as in “I am satisfied,” while xFD denotes “respondent will [future deed],” as in “I intend to purchase again.” The complete notation key is given in Appendix A.

Because each concept admits only its licensed structure codes, the pair (concept, structure) is a constrained label rather than a free choice. Table II lists all 22 concepts, their type, and the structure codes the framework permits for each. This table doubles as the reference the automated system is scored against.

B. Task Definition

Given the framework, we define two prediction tasks that compose into the full item-construction pipeline.

Assertion Development. The first task maps an indicator to a labeled declarative sentence:

$$f_A : i \mapsto (c, s, a), \quad (1)$$

where $c \in \mathcal{C}$ is the basic concept, $s \in \mathcal{S}(c)$ is a licensed structure code for that concept, and a is the assertion, a single declarative sentence that expresses c in structure s (for example, “I am satisfied with this product”).

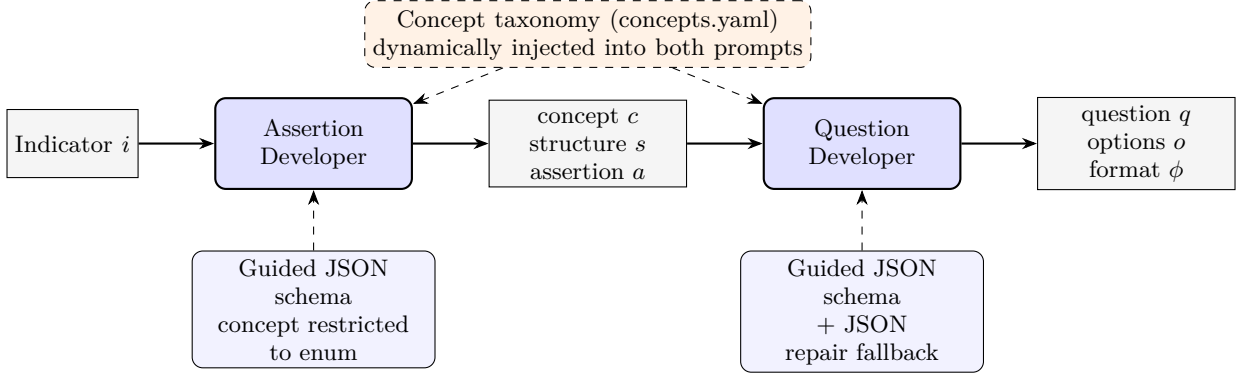


Fig. 1: The two-agent pipeline. An indicator is mapped by the Assertion Developer to a concept, structure code, and assertion; the Question Developer then turns an assertion into a question, options, and format. The concept taxonomy is injected into both system prompts at run time, and both stages use schema-guided JSON decoding, with the concept field constrained to the enumerated taxonomy.

Question Development. The second task transforms an assertion into a respondent-facing item:

$$f_Q : a \mapsto (q, o, \phi), \quad (2)$$

where q is the survey question, o is a set of answer options, and $\phi \in \Phi$ is the question format. The framework defines five formats $\Phi = \{\text{direct interrogative with a WH word, direct interrogative without a WH word, direct imperative, indirect interrogative, indirect imperative}\}$.

Each gold item provides the tuple $(i, c^*, s^*, a^*, q^*, o^*)$, where starred symbols denote the expert reference. This lets us score the concept and structure predicted by f_A against c^* and s^* , and the question produced by f_Q against q^* , as detailed in Section V. Following the framework’s own evaluation logic, the two tasks are assessed independently: f_Q is applied to the gold assertion a^* rather than to the predicted assertion a , so that errors in the first stage do not propagate into the measurement of the second.

IV. Method: The Two-Agent Pipeline

We implement the two tasks of Section III as two prompted agents connected in sequence, an Assertion Developer that realizes f_A and a Question Developer that realizes f_Q . Both agents are the same underlying open-weight model queried with different system prompts and output schemas; neither is fine-tuned. Figure 1 shows the overall architecture.

A. Dynamic Framework Injection

Rather than hard-coding the taxonomy into the prompts, we store it in a single machine-readable file (`concepts.yaml`) that lists every concept, its type, and its licensed structure codes, together with the notation key. At run time a prompt loader renders this file into a text block and substitutes it into a placeholder in each agent’s system prompt. Consequently the prompts always reflect the current taxonomy, and any change to the framework, such as correcting a licensed code, propagates to both agents without manual prompt edits. This coupling proved important during prompt engineering (Section VI), where aligning the reference file with the gold labels immediately changed model behavior.

B. Assertion Developer

The Assertion Developer receives the injected taxonomy and a short instruction to produce an assertion for a given indicator. Its system prompt is organized into four parts: a role description, a set of strict rules (identify exactly one concept, choose only a licensed structure code, output a single declarative sentence, never invent labels), a table of commonly confused concepts with disambiguation rules, and a set of worked examples. The disambiguation table encodes expert heuristics that separate frequently conflated categories, for instance Preference versus Expectations of future events, Action tendencies versus Behaviour, or Place versus Demographics, and it grew over successive prompt iterations as new confusions were observed. The agent returns a JSON object with the fields `concept`, `structure_code`, and `assertion`. The raw concept string is then mapped to its canonical taxonomy name before scoring, which absorbs superficial spelling differences.

C. Question Developer

The Question Developer receives an assertion and converts it into a respondent-facing item. Its prompt instructs the model to prefer clear, neutral, direct-interrogative phrasing, to choose a response format appropriate to the concept type (rating scales for subjective concepts; frequency scales, multiple choice, or open-ended formats for objective concepts), and to preserve the meaning of the assertion. It returns a JSON object with the fields `question`, `answer_options`, and `question_format`. To keep parsing robust, answer options are required to be a single delimited string rather than a nested list, which avoids a class of malformed-JSON failures observed early in development.

D. Structured Decoding and Robustness

Both agents use schema-guided JSON decoding: the server is given a JSON schema for the current stage and constrains generation to conform to it. For the Assertion Developer we additionally restrict the `concept` field to an enumeration of the 22 taxonomy names, which makes it impossible for the model to emit an out-of-taxonomy label and removes an entire error mode. A separate schema is used for the Question Developer and, later, for the judge. Two robustness measures handle the remaining failure cases. First, because the base model is a reasoning-capable model that may otherwise emit deliberation text, we disable its thinking mode and append an explicit instruction to return only JSON, which is necessary for reliable structured output. Second, a permissive parser strips any stray text around the JSON object, and a targeted repair routine reconstructs the question fields when the model breaks JSON quoting in the `answer_options` field.

E. Model and Serving

Both agents are served by a single instance of Qwen/Qwen3.5-9B, an open-weight instruction-tuned model, hosted locally through a high-throughput inference server exposing an OpenAI-compatible API [11], [12]. Generation is deterministic (temperature 0) with a token budget of 1024 per call, so that repeated runs of the same prompt are reproducible. The external judge used in Section V is a larger model, served separately on two GPUs with tensor parallelism. Two external judges appear in this paper: Qwen/Qwen2.5-72B-Instruct for the earlier judge run, and Qwen/Qwen3-32B for all later ones. The change was forced rather than chosen: the 72B checkpoint was removed from the shared model store partway through the project. To keep the substitution measurable we re-scored an identical set of predictions with both judges, and we report that comparison in Section VII.

Serving open weights locally keeps all data on-premise and fixes the model version across runs; the deployment ran on a single high-memory GPU, with the practical configuration details and workarounds recorded in Appendix F.

V. Experimental Setup

A. Gold Set

We evaluate against a hand-built gold set of 113 items constructed by expert annotation following the framework.¹ Each row specifies an indicator together with its reference concept, structure code, assertion, question, and answer options, and is additionally tagged with an application domain and a subjective/objective marker. Every one of the 22 basic concepts is represented, so the set exercises the full taxonomy rather than only its common categories. Of the 113 items, 71 realize subjective concepts and 42 realize objective concepts. The items are drawn from roughly 40 application domains, the most frequent being customer experience, health, employee experience, psychology and education, and social services; the long tail contains many domains represented by a single item, which tests generalization across topics rather than memorization of a narrow area. Table XI in Appendix C reports the per-concept counts.

The distribution is deliberately broad but uneven: frequent concepts such as Evaluation ($n=19$) and Demographics ($n=15$) are well sampled, whereas a number of concepts, including Norms, Policies, Values, and Causal relationship, appear only three times each. These rare categories yield high-variance per-concept estimates, a caveat we return to in Section IX, but they are essential for surfacing systematic failure modes on the harder parts of the taxonomy.

B. Isolated Evaluation Protocol

As motivated in Section III, the two agents are scored independently. The Assertion Developer is run on the gold indicator, and its predicted concept and structure are compared with the reference. The Question Developer is run on the gold assertion rather than the assertion just produced by the first stage, so that a mistake in concept or structure identification cannot contaminate the measurement of question quality. This isolated protocol mirrors the evaluation logic of the framework itself and yields per-stage metrics that are directly attributable to each agent.

C. Normalization

Objective scoring is exact match after a light normalization that removes differences irrelevant to the framework. Strings are lowercased and stripped of surrounding and internal whitespace before comparison. Concept names are additionally mapped across British and American spelling variants, so that behavior and behaviour, or judgment and judgement, are treated as identical.² Structure codes are compared after extracting the bare code token with a regular expression, which tolerates incidental annotation such as trailing qualifiers. Crucially, normalization is applied only in the scoring layer; the gold data itself is never modified.

¹The set was originally built with 115 items. Two were later found to be exact duplicates of other rows, identical in indicator, concept and structure, and were removed; one of the two also appeared verbatim as a worked example in an early prompt, so retaining it would have counted a memorized item twice. A third correction standardized the notation of one Evaluative-belief item. All reported numbers use the corrected 113-item set, and all earlier runs were re-scored against it.

²The reference file uses British spellings (Behaviour, Cognitive judgement) while the gold set uses American spellings; the normalization layer reconciles them without editing the source data.

D. Metrics

We report two families of metrics. The objective metrics measure adherence to the framework:

- Concept accuracy: fraction of items whose predicted concept matches the reference after normalization.
- Structure accuracy: fraction whose predicted structure code matches the reference.
- Both correct: fraction for which concept and structure are simultaneously correct, the strictest assertion-stage criterion.
- Conditional structure accuracy: structure accuracy restricted to items whose concept was identified correctly, $P(s \text{ correct} \mid c \text{ correct})$. Because most concepts license only one structure code (Table II), unconditional structure accuracy largely re-reports concept accuracy; the conditional form isolates what the structure prompt actually contributes and is the figure we treat as primary.
- Question coverage: fraction of items for which a non-empty question is produced, and the fraction for which a question format is assigned.
- Question exact match: fraction whose generated question matches the reference question string after normalization, reported as a strict lower bound on question quality.

The semantic metrics use an LLM as a judge to rate two alignments on a 1–5 scale: indicator-to-assertion (IA), whether the assertion faithfully represents the indicator, and assertion-to-question (AQ), whether the question faithfully represents the assertion. The judge is prompted with an explicit rubric (5 = perfect alignment; 4 = minor wording differences; 3 = partial; 2 = poor; 1 = none) and returns a score with a brief rationale, decoded under the judge JSON schema. Judge metrics complement exact match because a generated question may differ in wording from the reference yet still be a faithful, high-quality item.

E. Judge Models

We use two judge configurations. The self-judge is the same 9B model that generates the outputs, which is convenient but risks the self-preference bias discussed in Section II. The external judge is a larger, independently served model run on two GPUs: Qwen2.5-72B-Instruct for the first judge run, and Qwen3-32B thereafter, the substitution being forced by the removal of the 72B checkpoint from the shared model store partway through the project. The external judge re-scores the previously generated predictions without any change to prompts or generations, so any difference in scores is attributable to the evaluator rather than to the items being evaluated. Reporting both lets us compare evaluators directly. Because a difference between a small self-judge and a large external judge confounds who is judging with how capable the judge is, we additionally run a two-by-two control in which each of the two models judges both models’ generations; only that design separates self-preference from judge strictness.

F. Reproducibility

All runs are driven by a single configuration file that fixes the model paths, decoding parameters (temperature 0, 1024 tokens), the judge settings, and the evaluation scope. Because decoding is deterministic, re-running a configuration reproduces its outputs. Each reported run is tied to a specific code state, and the objective summaries and confusion matrices are exported as machine-readable artifacts alongside a per-item prediction table for manual inspection.

TABLE III: Interventions Introduced at Each Run (Carried Forward Thereafter)

Intervention	R1	R2	R3	R4	R5	R6	R7a	R7b	R7c
Isolated eval; JSON, thinking disabled	✓								
US/UK spelling-normalized scoring		✓							
Concept enum constraint (guided JSON)		✓							
Concept disambiguation table + examples		✓							
Structure disambiguation table + examples			✓						
Question JSON repair fallback			✓						
Confusion matrices + per-category metrics			✓						
Self-judge scoring (9B)				✓					
External judge scoring (72B)					✓				
Structure prompt pass 2						✓			
Reference file repaired						✓	✓		
Worked examples de-leaked (no gold verbatim)								✓	✓
Notation-derived deontic/causal rule									✓
External judge scoring (32B)									✓
Two-by-two generator × judge control									✓

VI. Prompt-Engineering Iterations

Rather than tuning the model, we improved the pipeline through a sequence of controlled runs, holding the rest of the system fixed at each step. We are careful about what this permits us to attribute. Runs 1 through 6 each grouped a small number of related changes, and Run 2 in particular combined a scoring change with two prompt changes, so their individual contributions cannot be separated from the run sequence alone. Run 7 remedies this: its three arms differ by exactly one intervention each and are evaluated on identical inputs. Table III records which technique entered at which run; once introduced, a technique is carried forward.

Run 1 (initial baseline). The first full run established the harness: isolated evaluation over the full gold set, schema-guided JSON decoding with the model’s thinking mode disabled, and exact-match scoring with only light string normalization. The assertion prompt contained a role, strict rules, and three worked examples. This run exposed three recurring problems that motivated the next iterations: concept mismatches caused purely by spelling, frequent confusion between related subjective concepts, and occasional malformed question JSON.

Run 2 (fairer scoring and concept constraints). Three changes targeted concept identification. Scoring was made spelling-invariant across British and American variants; the predicted concept field was constrained to the enumerated taxonomy so the model could no longer invent labels; and the assertion prompt gained a table of commonly confused concepts with disambiguation rules and three additional worked examples covering those cases. Together these changes chiefly improved concept accuracy.

Run 3 (structure disambiguation and richer diagnostics). Attention shifted to structure codes, which lagged concept accuracy. The prompt’s brief structure hints were expanded into a full disambiguation table with three further examples, most notably fixing the default structure for Preference. A repair routine recovered question fields when the model broke JSON quoting, eliminating the malformed-output cases. This run also added the diagnostic exports, per-concept and per-structure breakdowns and confusion matrices, that the analysis in Section VIII relies on.

Run 4 (self-judge). No prompts or generations changed; the run only enabled LLM-as-judge scoring using the same 9B model, adding the IA and AQ semantic scores. This isolates the effect of adding a semantic metric from any change in the items themselves.

Run 5 (external judge). The Run 4 predictions were re-scored by the larger 72B judge, again without regenerating any items. Comparing Run 4 and Run 5 scores on identical outputs isolates the effect of the evaluator, though, as Run 7 shows, it does not by itself identify that effect as self-preference.

Run 6 (structure prompt pass 2). The final iteration addressed the structure codes that remained systematically wrong after Run 3. Two worked examples that had taught incorrect codes were corrected, the disambiguation table was extended with rules for feelings, place and procedures, quantities, time, and the values-versus-importance distinction, and seven new examples were added. Critically, the machine-readable reference file was aligned with the gold structure codes, which, through the dynamic injection of Section IV, immediately propagated the corrected codes into the prompt. This run produced the largest single-iteration gain in structure accuracy. It also, as we discuss in Section IX, wrote gold-derived structure codes into the prompt, which is one of the reasons Run 7 re-examines the prompt with the worked examples removed.

Run 7 (ablation, judge replacement, and controls). The preceding runs bundle several changes each and were all tuned by inspecting errors on the same items they are scored on. Run 7 therefore varies one thing at a time across three arms evaluated on identical inputs. Arm R7a repeats the Run 6 prompt with only the repaired reference file, isolating that repair. Arm R7b additionally replaces every worked example with a synthetic one, so that no gold item appears verbatim in the prompt; this arm exists to measure how much of the reported accuracy depended on items the prompt had memorized. Arm R7c adds a single rule, derived from the notation rather than from inspecting errors, that distinguishes the deontic subject symbols underlying Norms and Policies from the general-subject symbol underlying Values, and separates causal from similarity relations. Run 7 also re-scores predictions with a replacement external judge and runs the two-by-two generator-by-judge control described in Section V.

VII. Results

We report objective and semantic results for the runs of Section VI. Unless noted, “best” refers to arm R7c, our strongest configuration and the only one that combines a de-leaked prompt with the notation-derived rule.

A. Overall Progression

Table IV and Fig. 2 summarize the assertion-stage metrics across runs. Prompt engineering alone, with no fine-tuning, raises concept accuracy from 65.5% to 85.0% and structure accuracy from 51.3% to 79.6%; the strict both-correct criterion rises from 47.8% to 79.6%. All runs are scored on the same 113 items under the same normalizer, so the columns are directly comparable, and Run 4 is omitted from the table because it reproduces the Run 3 generations exactly and adds only judge scores.

Two qualifications matter more than the trend line. First, because every run is evaluated on the identical item set, the comparisons are paired, and an exact McNemar test is the appropriate instrument. Applying it to each consecutive transition (Table V) shows that only two of the six are significant at the 5% level: the structure pass in Run 6 ($p = 0.024$) and the notation rule in Run 7c ($p = 0.039$). The remaining transitions, including the Run 3 to Run 6 concept gain that earlier drafts of this work described as producing the best configuration, are within sampling noise at this sample size. Second, the largest early gain is partly an artifact of measurement rather than of generation. Run 2 introduced a spelling-normalized scoring layer at the same time as the enum constraint and the disambiguation table; re-scoring the Run 1 outputs under that normalizer, with no change to the generations, raises Run 1 from 57.5% to 65.5%. Of the 11.5-point Run 1 to Run 2 concept gain, 8.0 points are therefore attributable to the change in the measuring instrument, and the residual paired comparison is not significant ($p = 0.503$). We report the normalizer-adjusted figure throughout so that the progression reflects model behavior rather than scoring changes.

TABLE IV: Objective and Semantic Metrics Across Runs (Percent, or 1–5 for Judge Scores)

Metric	R1	R2	R3	R6	R7a	R7b	R7c
Concept accuracy	65.5	69.0	75.2	76.1	78.8	77.9	85.0
95% CI	56–74	60–77	67–82	68–83	70–85	69–85	77–90
Structure accuracy	51.3	54.0	61.1	74.3	76.1	73.5	79.6
95% CI	42–60	45–63	52–70	66–82	68–83	65–81	71–86
Both correct	47.8	51.3	59.3	72.6	74.3	71.7	79.6
Question coverage	99.1	99.1	100	100	100	100	100
Question exact match	20.4	20.4	18.6	17.7	20.4	20.4	20.4

All values on the corrected 113-item gold set. R7a–R7c are the three ablation arms of Run 7, each adding exactly one intervention. Run 4 is omitted: it re-scores the Run 3 generations unchanged.

TABLE V: Exact McNemar Tests on Consecutive Runs (Paired, $N = 113$)

Transition	Concept		Structure	
	b/c	p	b/c	p
R1 → R2	8/12	0.503	7/10	0.629
R2 → R3	3/10	0.092	3/11	0.057
R3 → R6	12/13	1.000	12/27	0.024
R6 → R7a	0/3	0.250	1/3	0.625
R7a → R7b	2/1	1.000	4/1	0.375
R7b → R7c	2/10	0.039	2/9	0.065

b = items correct before but wrong after; c = wrong before, correct after.

Only two transitions reach $p < 0.05$.

B. Per-Concept and Per-Structure Behavior

Three further measurements put the headline number in context.

Reference points. The pipeline is well clear of trivial baselines but not of a larger model by default. A majority-class classifier that always predicts the most frequent concept reaches 16.8%, and a leave-one-out lexical nearest-neighbour classifier over TF-IDF representations of the indicators reaches 32.7% concept and 31.0% structure accuracy. More informative is the upper reference point: running the identical Run 7c prompt through Qwen3-32B, a model roughly $3.5\times$ larger, yields 77.0% concept and 68.1% structure accuracy, below the 9B pipeline on both (Table VI). The paired comparison is directional rather than conclusive ($p = 0.064$), but it indicates that the residual errors are not primarily a matter of model capacity, and that framework adherence is not something scale supplies for free.

Structure is largely determined by concept. Conditional on a correct concept, structure accuracy is 93.8% (90/96); when the concept is wrong, it is 0.0% (0/17). Under the best configuration the model never recovers the correct structure code after misidentifying the concept. Unconditional structure accuracy and the both-correct criterion therefore largely re-report concept accuracy, and we regard $P(s | c)$ as the informative structure metric. A practical consequence is that concept errors are worth twice their nominal cost.

How much of earlier accuracy was memorized. Sixteen of the original gold items appeared verbatim, with their labels, as worked examples in the prompt used through Run 6; fifteen survive into the corrected item set. Splitting the R7a arm, which still uses that prompt, on this boundary gives 15/15 concept accuracy on the quoted items against 75.5% (74/98) on the rest (Table VII). Every quoted item receives the correct concept, and all but one the correct structure. Arm R7b removes them, and its accuracy on the remaining items is statistically indistinguishable from R7a’s ($p = 1.0$), so the prompt was not depending on those examples in aggregate even though it had memorized them individually. We report R7b and R7c, which contain no verbatim gold items, as the configurations whose numbers are not inflated by this effect.

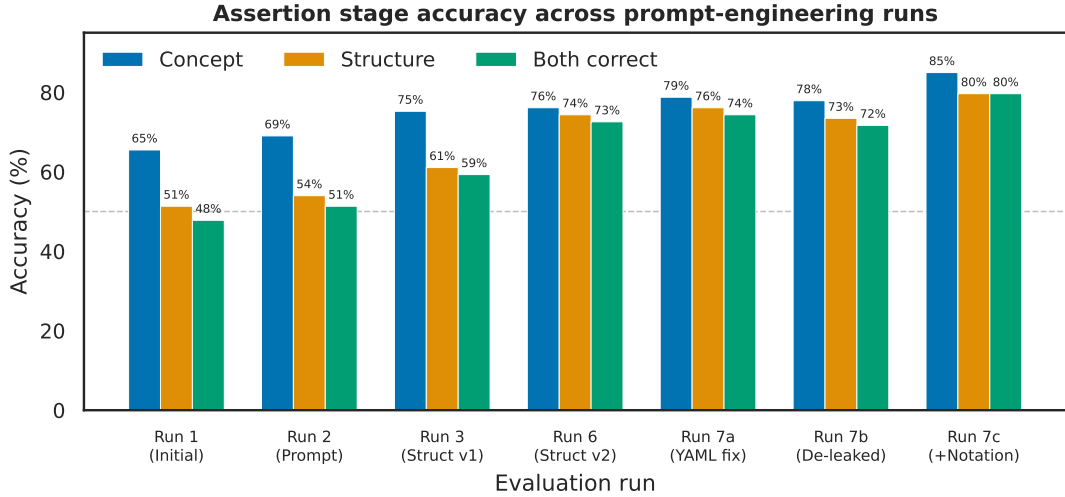


Fig. 2: Assertion-stage accuracy across the seven prompt-engineering runs (the judge-only runs are omitted). Prompt engineering alone moves concept accuracy from 65.5% to 85.0% and structure accuracy from 51.3% to 79.6% on the corrected 113-item gold set. The dip at R7b is the de-leaking of the worked examples, which is not statistically significant.

TABLE VI: Reference Points for Concept and Structure Accuracy ($N = 113$)

System	Concept	Structure
Majority class	16.8	—
Lexical 1-NN (TF-IDF, leave-one-out)	32.7	31.0
Qwen3-32B generator, Run 7c prompt	77.0	68.1
Qwen3.5-9B pipeline (Run 7c)	85.0	79.6

Aggregate accuracy hides strong variation across the taxonomy. Fig. 3 shows per-concept results at the best configuration, and Table XI gives them as counts. Nine concepts, namely Causal relationship, Expectations of future events, Importance, Norms, Rights, Knowledge, Place, Quantities, and Time, are answered correctly on every item for both concept and structure. The frequent categories sit somewhat lower: Evaluation at 17/19, Demographics at 13/15, Preference at 7/9. The weakest cells are Events (2/5) and Evaluative belief, which is identified correctly on all three items but scores 0/3 on structure, for a reason that is a property of the gold notation rather than of the model and which we return to in Section IX. We stress that the three-item cells should not be read as percentages. Norms and Policies, which motivated the Run 7c intervention, move from 0/3 and 0/3 before the rule to 3/3 and 2/3 after it; that is a real and targeted improvement, but it rests on five items, and the paired test over the full item set ($p = 0.039$) is the statistic we rely on rather than the per-cell counts.

A recurring pattern is a gap between getting the concept right and getting its structure right (Fig. 6, Appendix C). Run 6 largely closed the gap for several codes that had been systematically wrong in Run 4: the future-deed code rose from 0% to 88%, the place code from 0% to 83%, the feeling code from 0% to 75%, and the time, quantity, and societal-value codes each reached 100%. What remains is concentrated in a few categories where a correct concept is still paired with the wrong structure, discussed in Section VIII.

C. Question Stage

The Question Developer solves coverage: from Run 3 onward it produces a non-empty question and an assigned format for every item (100%). Exact string match against the

TABLE VII: Accuracy Split by Whether the Item Appears Verbatim in the Prompt (Arm R7a)

Subset	n	Concept	Structure
Quoted in the prompt	15	15/15	14/15
Not quoted	98	74/98	72/98
All items	113	89/113	86/113

Arms R7b and R7c quote no gold items and so admit no such split.

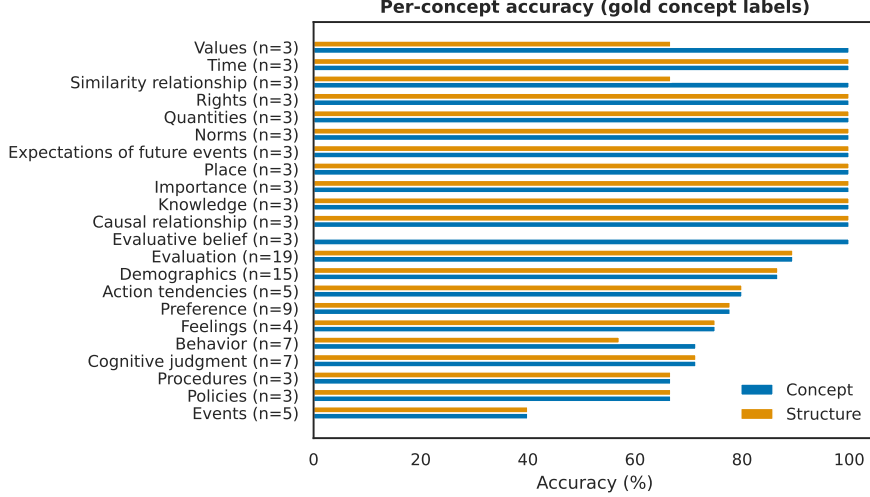


Fig. 3: Per-concept concept and structure accuracy at the best configuration (R7c). Errors concentrate on Events and on Evaluative belief, whose structure code is scored zero for a notational reason discussed in Section IX; the deontic concepts that dominated earlier runs are repaired. Cells with $n=3$ should be read as counts (Table XI), not percentages.

reference question, however, is low and flat across runs (20.4% at Run 7c, and never above 20.4% anywhere). This is expected: many faithful questions are simply worded differently from the reference. Fig. 4 contrasts exact match with judge scores and shows that the two are essentially uncorrelated, so exact match understates question quality; the semantic results below make this precise.

Two of the three fields the Question Developer emits deserve a more critical reading than earlier drafts of this work gave them. The question format ϕ is degenerate: every one of the 113 generated items is tagged as a direct interrogative with a WH word, so ϕ takes a single value across the corpus and the five-format space Φ of Section III is unused. A field with no variance cannot discriminate between systems, and we do not treat it as a result; it reflects the prompt’s stated preference for that form rather than a property of the items. The answer options o were not scored against the reference at all in earlier drafts, despite being part of the gold tuple. Scoring them by response type, that is, comparing whether the generated and reference options are both ordinal scales, both binary, both nominal, both ranked, or both open-ended, rather than requiring string identity, gives 61.1% agreement. That is markedly weaker than the question stage itself and identifies answer-option generation as the least reliable part of the pipeline, and the part least examined by our evaluation.

D. Semantic (Judge) Results

Semantic scoring confirms that both stages are strong. Under the self-judge, mean indicator-to-assertion alignment is 4.51/5 and mean assertion-to-question alignment is

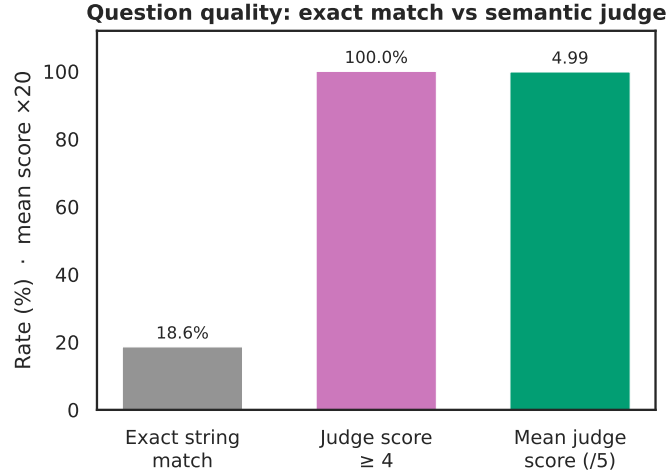


Fig. 4: Question quality by exact match versus LLM-as-judge. The large majority of questions that do not match the reference wording are nonetheless rated highly aligned, so exact match severely understates quality.

TABLE VIII: Two-by-Two Generator $\times$ Judge Control: Mean Indicator-to-Assertion Score ($N = 113$)

Generator	Judged by 9B	Judged by 32B	Self – other
Qwen3.5-9B	4.48	4.43	−0.01
Qwen3-32B	4.49	4.39	−0.04

Self-preference would appear as a positive value in the last column.

Both are negative: each judge is marginally harsher on its own output.

4.99/5. Re-scoring the identical predictions with the external Qwen2.5-72B-Instruct judge lowers both means, to 4.09 for IA and 4.56 for AQ, drops of 0.42 and 0.43 points respectively. Despite this correction, the question stage remains strong under the stricter judge: every item scores at least 4 on assertion-to-question alignment, including all 94 questions that do not exactly match the reference. An assertion-to-question mean of 4.99/5 from the self-judge is, however, better read as a degenerate measurement than as a quality result, a point we develop below.

The judge comparison above confounds two things, because the two judges differ in identity and in capacity. A larger model being stricter is as consistent with the data as a model preferring its own outputs. We therefore ran the two-by-two control, in which each of the 9B generator and a Qwen3-32B model judges both models’ generations on the same 113 items (Table VIII). The result does not support a self-preference reading. Each judge scores the two generators within 0.05 points of each other, and the 32B judge is in fact marginally harsher on its own output than on the 9B’s. What separates the judges is not whose output they are grading but how they use the scale. On identical generations the two means differ by only 0.05, yet the 9B judge places 1% of items in the middle of its five-point scale against 38% for the 32B judge, and fails to return a parseable score on three items altogether (Fig. 5). The 9B judge emits a 5 or a 2 and almost nothing else. We conclude that the earlier self-grading result is better described as a difference in judge quality than as self-preference, and that reporting a judge’s score distribution is necessary: the means alone would have made these two instruments look equivalent.

The same 113 generations, judged by two models

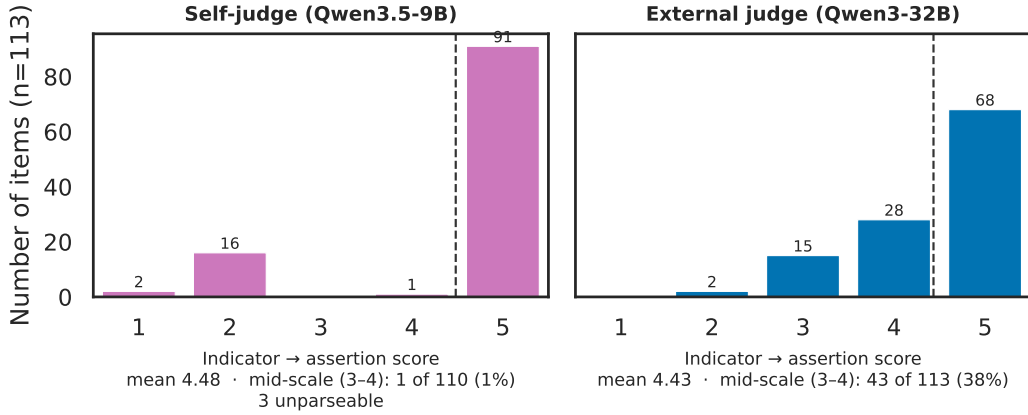


Fig. 5: The same 113 generations scored by the 9B self-judge and the 32B external judge. The means are nearly identical (4.48 against 4.43), but the distributions are not comparable: the self-judge assigns 1% of items to the middle of the scale against 38% for the external judge, and returns three unparseable scores. Reporting means alone would conceal this.

VIII. Error Analysis and Discussion

We now examine where the pipeline still fails and what the results imply for automated survey item construction. The confusion structure is analyzed from the per-item predictions of the best configuration; the full confusion matrices and the most frequent confusing pairs (Fig. 9, Fig. 10) are given in Appendix D.

A. Concept Confusions

The remaining concept errors are not random but cluster on a few semantically adjacent pairs (Fig. 9, Appendix D). The two most frequent are Cognitive judgment read as Events, and Events read as Feelings; Events are also confused with Demographics when a past occurrence is phrased as a static status, for example a medical condition read as a demographic attribute, and Preference is confused with Evaluation and with Feelings when an indicator mixes an appraisal with a desired change. No single pair accounts for more than two items. Notably, the Norms-versus-Values confusion that dominated the earlier runs is absent at this configuration: the deontic rule of Section VI removed it entirely, and what remains are distinctions between adjacent objective categories rather than the deontic cluster. These are broadly the distinctions that expert annotators also find hardest, which is one reason we regard the absence of a reported inter-annotator agreement (Section IX) as a real gap rather than a formality.

B. Structure Confusions and the Concept–Structure Coupling

Structure errors have two distinct sources (Fig. 10, Appendix D). The first is coupling: when the concept is wrong, the structure is almost always wrong too, because the licensed codes differ between concepts. The second, more interesting, source is cases where the concept is correct but the structure is not. Before Run 6 this was systematic: Expectations of future events reached full concept accuracy while its structure code was consistently wrong, and similar behavior appeared for action tendencies and feelings. Aligning the

TABLE IX: The Norms/Values Trade-Off Across Runs (Correct Items, $n=3$ Each)

Concept	R1	R2	R3	R6	R7a	R7b	R7c
Norms	3/3	3/3	3/3	0/3	1/3	0/3	3/3
Policies	0/3	0/3	0/3	0/3	0/3	0/3	2/3
Values	1/3	2/3	1/3	3/3	3/3	3/3	3/3
Concept identification. Run 6 gains Values and loses Norms; Run 7c holds both.							

reference file with the gold codes and correcting two misleading examples removed most of these errors at once. What remains are a handful of within-concept structure slips, for example the evaluative-belief predicate code, the values-versus-importance code, and the similarity code, each affecting only one or two items.

C. A Prompt-Engineering Trade-off: The Norms Regression

Prompt engineering is not monotonic. The Run 6 pass that strengthened the values-versus-importance rules had a side effect: the model began mapping deontic “should” and “ought” indicators to Values rather than Norms. Table IX traces the three affected concepts across every run and makes the trade-off explicit. Norms is answered correctly on all three items in Runs 1 through 3, collapses to 0/3 in Run 6, and recovers to 3/3 only in Run 7c; Values moves in the opposite direction over the same interval, from 1/3 to 3/3. The Run 6 prompt did not simply fail on Norms, then: it purchased Values accuracy at the cost of Norms accuracy, because both categories are cued by the same modal surface forms.

The Run 7c fix is not another disambiguation example. The framework already distinguishes the two: concepts.yaml assigns Norms the general-subject symbol o and Policies the government symbol g , while Values uses v . The prompt’s tables named the concepts but never explained those symbols, so the distinction existed in the notation and was invisible to the model. Adding one rule that derives the concept from the subject symbol repairs Norms without disturbing Values, and moves Policies from 0/3 to 2/3, a category that had been wrong on every item in every previous run. We note the shape of this result: the most effective single intervention we found was not new examples but a statement of a distinction the framework already encoded.

D. Self- versus External-Judge Bias

The gap between the two judges is not uniform across the taxonomy. The largest self-minus-external differences cluster on Demographics indicators, where the external judge penalizes first-person paraphrases (for example, rendering an employment-status indicator as “My employment status is...”) as a scope mismatch for what should be a neutral factual item, whereas the self-judge rated them perfect. As the two-by-two control of Section VII shows, this is not self-preference: with generator and judge varied independently, neither model favors its own output. The defensible conclusion is narrower and more useful. A small self-judge is an unreliable instrument irrespective of whose text it grades, because it does not use its own rating scale; an independent judge should be reported not because self-evaluation is self-serving, but because a weak judge cannot discriminate.

E. Exact Match Understates Question Quality

The question stage exposes a measurement pitfall. Exact string match against the reference is low (20.4%) and essentially uncorrelated with judged alignment: under the external judge, every non-matching question scored at least 4 on assertion-to-question alignment. Exact match here measures wording overlap rather than quality and would

badly misrank the system, so we recommend judge-based alignment as the primary question metric, with exact match reported only as a conservative lower bound.

F. Prompt Engineering versus Fine-Tuning

Framework-grounded prompt engineering thus reaches 85.0% concept and 79.6% structure identification, with near-ceiling semantic quality on the question stage, without any parameter updates, while a $3.5\times$ larger generator on the same prompt does worse. Fine-tuning is therefore better seen as an optional refinement for the long tail of rare, semantically subtle concepts than as a prerequisite for a strong baseline.

IX. Limitations

Several limitations bound the scope of our conclusions, and we state the most serious one first.

All reported numbers are in-sample. Every prompt in this paper was written or revised by inspecting errors on the same items the results are computed over. Runs 1 through 6 additionally quoted fifteen gold items verbatim, with their labels, and Run 6 wrote gold structure codes into the injected reference file; we measured the effect directly (Table VII), and report the de-leaked arms R7b and R7c as headline results. But removing verbatim items does not make the evaluation held-out. The rules in the R7c prompt, including the deontic-subject rule that produces its largest gain, were still written by a person who had seen these items and their labels. The figures here should be read as measures of fit to a taxonomy on a known sample, not as estimates of generalization to new indicators. Establishing the latter requires items annotated after the prompts were frozen, which we did not have; we regard this as the most important piece of future work rather than a detail.

Gold-set corrections. Beyond the two duplicates removed in Section V, the gold labels used three different notations for the single concept Evaluative belief, and the injected reference file licensed a fourth. We standardized on the notation-derived form, which lowers our reported structure accuracy by about one point, because the model consistently produces one of the discarded variants and is now scored wrong on all three such items (Table XI). Standardizing in the opposite direction would have raised the reported figure by moving the reference toward what our own prompt teaches, which is why we did not do it.

No inter-annotator agreement. The gold set was constructed by expert annotation, but we do not report how many annotators contributed, their agreement, or how disagreements were resolved, and no subset was double-annotated. Every objective metric in this paper is exact match against these labels. This matters most exactly where the model fails: the Norms, Policies and Values cluster involves distinctions that expert annotators also find difficult, so an unknown share of the errors we attribute to the model may be label disagreements. A double-annotated subset with a reported κ would settle this and should accompany any future version of the gold set.

The remaining limitations are narrower. First, the gold set is small: 113 items, with fourteen of the twenty-two concepts represented by only three examples each. We report these cells as counts rather than percentages for that reason, and the targeted gain in Section VIII, though statistically significant over the full item set, rests on five items. Second, all generation uses a single open-weight model family; the judges are drawn from the same family, so cross-family disagreement may be understated. Third, our semantic evaluation relies on an LLM judge. Our two-by-two control finds no self-preference bias, but it does show that the smaller judge barely uses its rating scale, and we do not validate either

judge against expert human ratings; the absolute alignment scores should therefore be read as model-based estimates rather than ground truth. The judge substitution is a further confound: because the 72B checkpoint became unavailable partway through, the difference between the two external judges cannot be decomposed into model scale, model family, and reasoning-mode effects. Fourth, the evaluation scores each stage in isolation and on single-facet indicators; it does not measure end-to-end behavior on broad or compound indicators, nor downstream data-quality outcomes such as respondent comprehension or measurement validity. Finally, the gold set and prompts are English-only, and the framework’s concept and structure taxonomy, though general in intent, is applied here within that single-language setting.

X. Conclusion and Future Work

We presented a framework-grounded, two-agent pipeline that turns raw indicators into survey items by following a formal three-step design method, using constrained decoding to keep every output inside the concept-and-structure taxonomy. Evaluated against a 113-item expert gold set with a dual protocol, it reaches 85.0% concept and 79.6% structure accuracy through prompt engineering alone. Reporting Wilson intervals and paired McNemar tests throughout, we find only two of six run-to-run transitions individually significant, and a substantial share of the largest early gain attributable to a change in the scoring normalizer rather than to the model; we regard this as the more honest characterization of what iterative prompt engineering delivers. Along the way we showed that exact string match badly understates generative question quality, that a two-by-two generator-by-judge control finds no self-preference bias but does reveal a self-judge which uses only two points of a five-point scale, and that a $3.5\times$ larger generator performs worse on the identical prompt.

Four directions follow, ordered by how much they would change what can be claimed. The first is a genuinely held-out test set, indicators annotated after the prompts are frozen, which is the only way to convert the figures reported here from measures of fit into estimates of generalization; a double-annotated subset with a reported agreement coefficient should accompany it, so that model errors can be separated from label disagreements. Second, the judge should be validated against expert human ratings: our control establishes that a small judge is unreliable but cannot establish that the larger one is correct, and a methodologically-native instrument such as a survey-quality predictor would be a stronger reference still, because it does not share the failure modes of an LLM evaluator. Third, answer-option generation, the weakest component we measured at 61.1% response-type agreement, has received no prompt work at all; a smaller pending fix is to align the prompt’s structure tables with the standardized Evaluative-belief notation, which our current prompts still contradict. Fourth, parameter-efficient fine-tuning on the rare concepts is a natural refinement once prompt-based gains plateau. More broadly, the combination of taxonomy-grounded generation and a dual, bias-aware evaluation offers a reusable template for applying large language models to other structured measurement tasks in computational social science.

Appendix A Notation Key

Table X lists the symbols used to build the structure codes of Section III. A code concatenates a subject symbol, a linking element, and, where present, an object or complement symbol; the three base templates are given in Table I.

TABLE X: Structure-Code Notation

Symbol	Meaning
x	grammatical subject (respondent or topic)
r	respondent as actor
o	generic / anyone as subject
g	government as subject
I	link verb (is / are / was)
P	predicator (general)
C	causal relationship (subject causes object)
D	deed / action
F	feeling
FD	future deed
$H(+I)$	has to / should + infinitive
HR	has the right to
PR	preference
S	similarity or difference
e	evaluative adjective
i	importance adjective
f	feeling noun / adjective
c	cognitive judgement
ca	causal marker
s	similarity marker
pr	preference marker
ri	rights marker
d	demographic descriptor
sc	scale / classifier
ti	time expression
pl	place expression
qu	quantity expression
pro	procedure expression
v	value or ideal as subject
(e)	evaluative qualifier on the predicator, as in $xP(e)y$
y	object / second entity
z	optional comparison referent; licensed for Preference but not exercised by any gold item

Appendix B Agent Prompts

Both prompts begin with the dynamically injected taxonomy block (Section IV); only the fixed portions are reproduced here in abbreviated form.

Assertion Developer. Role: an expert survey methodologist that converts a raw indicator into a theory-grounded assertion. Strict rules: (1) identify exactly one basic concept, using its exact taxonomy name; (2) choose only a structure code licensed for that concept; (3) output a single declarative sentence following that structure; (4) keep the sentence natural and respondent-appropriate; (5) never invent concepts or codes; (6) apply the disambiguation rules when the indicator is ambiguous; (7) output only a JSON object with keys `concept`, `structure_code`, `assertion`. The prompt then supplies a “commonly confused concepts” table (for example, Evaluation vs. Evaluative belief; Expectations of future events vs. Action tendencies; Place vs. Demographics), a structure-disambiguation table keyed by code, and sixteen worked examples. A representative example: indicator “repurchase intention” maps to concept Action tendencies, code xFD , assertion “I intend to purchase from this organization again.”

Question Developer. Role: an expert questionnaire designer that turns an assertion into a question and answer options. Rules: convert the declarative assertion into a clear, neutral question; prefer direct-interrogative phrasing; select a response format appropriate to the concept type (rating scales for subjective concepts; frequency, multiple-choice, or open-ended for objective ones); preserve meaning; return answer options as a single

delimited string; and output only a JSON object with keys question, answer_options, question_format. A representative example: assertion “I am satisfied with this product” maps to question “How satisfied are you with this product?”, a rating scale, tagged as a direct interrogative.

Appendix C

Complete Per-Concept Results

Table XI reports concept and structure accuracy for all 22 concepts at the best configuration (Run 7c), as raw counts. Fig. 6 plots the same data as concept accuracy against structure accuracy, so that concepts the model labels correctly but still misstructures fall below the diagonal. Counts rather than percentages are used deliberately: fourteen of the twenty-two concepts have $n=3$, where a single item moves the percentage by 33 points and a Wilson interval spans most of the unit interval.

TABLE XI: Per-Concept Accuracy at the Best Configuration (Run 7c), as Counts

Concept	n	Concept	Structure
Subjective			
Evaluation	19	17/19	17/19
Preference	9	7/9	7/9
Cognitive judgement	7	5/7	5/7
Action tendencies	5	4/5	4/5
Feelings	4	3/4	3/4
Causal relationship	3	3/3	3/3
Evaluative belief	3	3/3	0/3
Expectations of future events	3	3/3	3/3
Importance	3	3/3	3/3
Norms	3	3/3	3/3
Policies	3	2/3	2/3
Rights	3	3/3	3/3
Similarity relationship	3	3/3	2/3
Values	3	3/3	2/3
Objective			
Demographics	15	13/15	13/15
Behaviour	7	5/7	4/7
Events	5	2/5	2/5
Knowledge	3	3/3	3/3
Place	3	3/3	3/3
Procedures	3	2/3	2/3
Quantities	3	3/3	3/3
Time	3	3/3	3/3

Evaluative belief scores 0/3 on structure because the gold notation for that concept was standardized after the run; see Section IX.

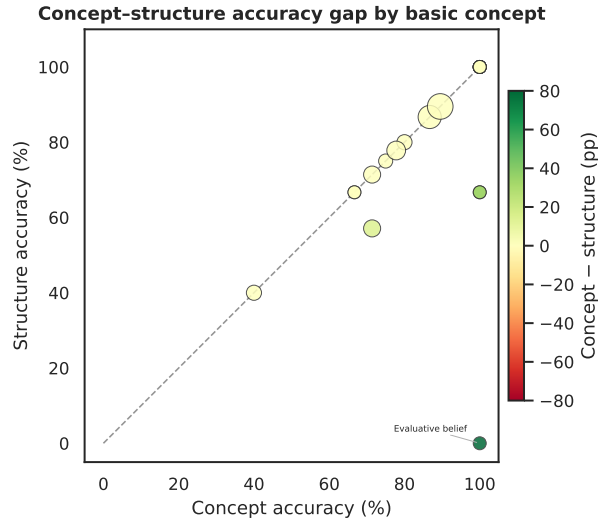


Fig. 6: Concept accuracy versus structure accuracy per concept. Points below the diagonal identify concepts the model labels correctly but whose semantic structure it still misassigns.

Appendix D Confusion Matrices

Fig. 7 and Fig. 8 give the full concept and structure confusion matrices at the best configuration; Fig. 9 and Fig. 10 extract the most frequent confusing pairs discussed in Section VIII.

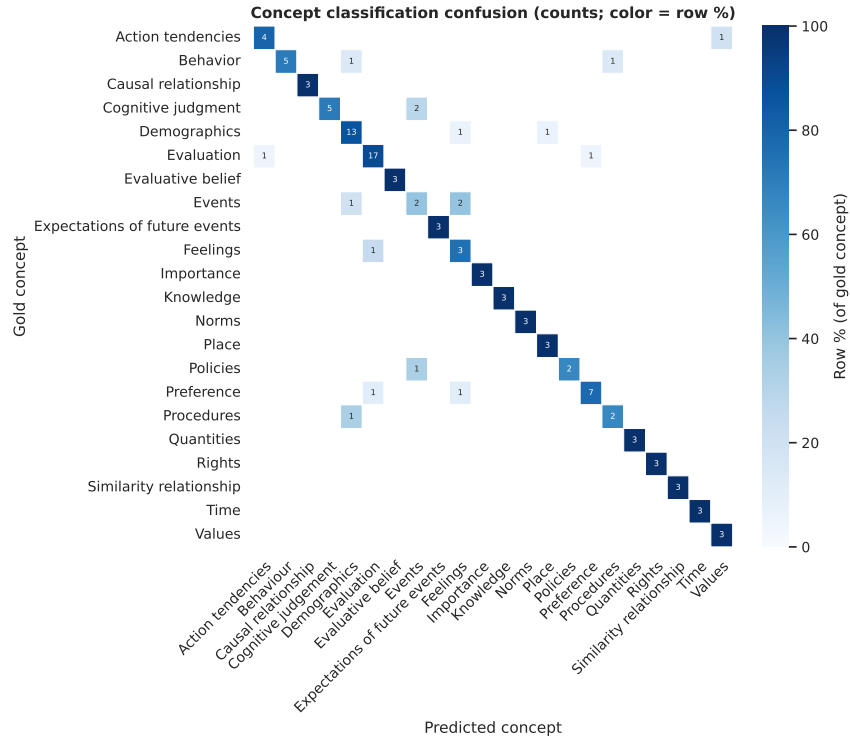


Fig. 7: Concept confusion matrix at the best configuration (R7c). Rows are gold concepts, columns are predictions; color encodes row-normalized rate.

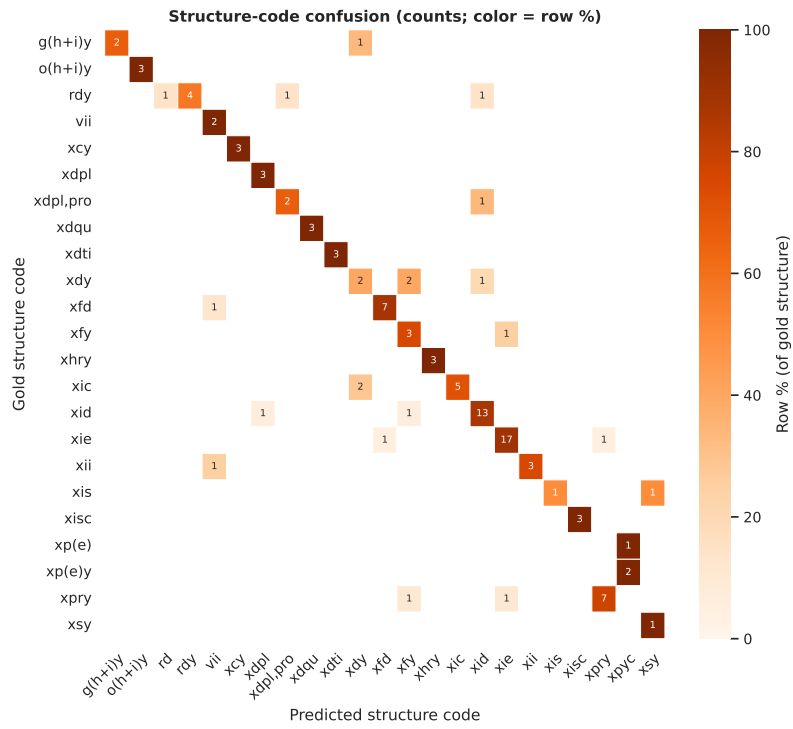


Fig. 8: Structure-code confusion matrix at the best configuration (R7c).

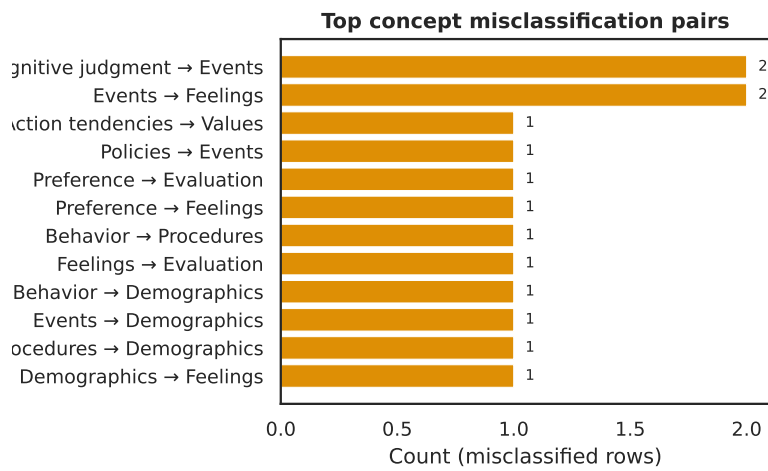


Fig. 9: Most frequent concept misclassification pairs at the best configuration. Errors concentrate on semantically adjacent categories rather than being spread uniformly.

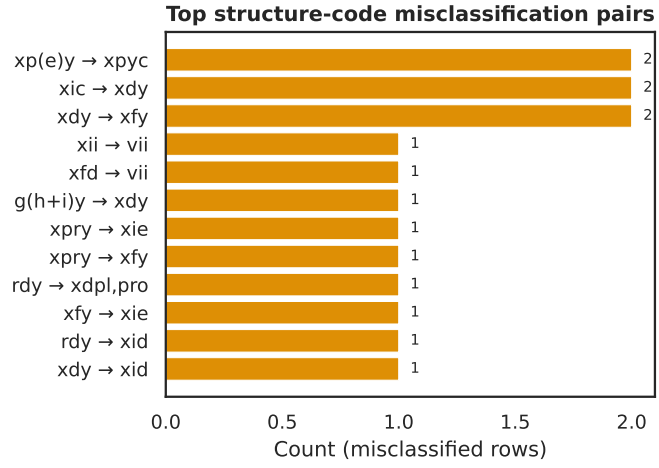


Fig. 10: Most frequent structure-code misclassification pairs. Many are induced by an upstream concept error; the rest are within-concept slips on a small number of codes.

Appendix E

Judge Rubric and Hardest Items

The judge is prompted with a fixed rubric on a 1–5 scale: 5 perfect alignment; 4 minor wording or format differences only; 3 partial alignment, related but missing nuance or scope; 2 poor alignment, significant mismatch; 1 no alignment. It returns a score and a brief rationale under the judge JSON schema. Table XII lists the items with the lowest external indicator-to-assertion scores, most of which coincide with concept errors and with first-person phrasings of factual indicators; the last row also illustrates the self-judge over-rating an item the external judge scored low.

TABLE XII: Hardest Items by External Indicator-to-Assertion Score

Indicator	Error	Ext	Self
Medication details	Procedures → Demographics	1	1
Average stress level	Feelings, wrong structure	2	2
Medical conditions	Events → Demographics	2	2
Areas for improvement	Evaluation → Preference	2	2
Length of engagement	Time → Demographics	2	5

Appendix F

Reproduction Details

Generation and judging use an OpenAI-compatible inference server hosting the open-weight models locally on GPU. The generator (Qwen/Qwen3.5-9B) runs on a single high-memory GPU; the external judges (Qwen/Qwen2.5-72B-Instruct, and Qwen/Qwen3-32B after the former was removed from the shared model store) run on two GPUs with tensor parallelism. Decoding is deterministic (temperature 0, up to 1024 tokens per call). For reliable structured output from the reasoning-capable base model, thinking mode is disabled and an explicit instruction to emit only JSON is appended to each request; both stages use schema-guided JSON decoding, and the assertion stage additionally constrains the concept field to the enumerated taxonomy. On the target hardware the server required a non-default attention backend and eager execution to run stably; the exact backend flags, library-path ordering, and launch scripts are provided with the code release. Runs are configuration-driven, so fixing the configuration file and model version reproduces the reported outputs, and each run additionally exports a per-item prediction table together with the objective summary and confusion matrices.

References

- [1] R. M. Groves, F. J. Fowler, M. P. Couper, J. M. Lepkowski, E. Singer, and R. Tourangeau, *Survey Methodology*. Hoboken, NJ, USA: Wiley, 2nd ed., 2009.
- [2] W. E. Saris and I. N. Gallhofer, *Design, Evaluation, and Analysis of Questionnaires for Survey Research*. Hoboken, NJ, USA: John Wiley & Sons, Inc., 2007.
- [3] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. M. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei, “Language models are few-shot learners,” 2020.
- [4] P. Liu, W. Yuan, J. Fu, Z. Jiang, H. Hayashi, and G. Neubig, “Pre-train, prompt, and predict: A systematic survey of prompting methods in natural language processing,” 2021.
- [5] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica, “Judging llm-as-a-judge with mt-bench and chatbot arena,” 2023.
- [6] G. Kurdi, J. Leo, B. Parsia, U. Sattler, and S. Al-Emari, “A systematic review of automatic question generation for educational purposes,” *International Journal of Artificial Intelligence in Education*, vol. 30, pp. 121–204, 2019.

- [7] C. Ziems, W. Held, O. Shaikh, J. Chen, Z. Zhang, and D. Yang, “Can large language models transform computational social science?,” 2024.
- [8] P. Wang, L. Li, L. Chen, Z. Cai, D. Zhu, B. Lin, Y. Cao, Q. Liu, T. Liu, and Z. Sui, “Large language models are not fair evaluators,” 2023.
- [9] A. Panickssery, S. R. Bowman, and S. Feng, “Llm evaluators recognize and favor their own generations,” 2024.
- [10] B. T. Willard and R. Louf, “Efficient guided generation for large language models,” 2023.
- [11] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, “Efficient memory management for large language model serving with pagedattention,” 2023.
- [12] A. Yang, A. Li, B. Yang, B. Zhang, B. Hui, B. Zheng, B. Yu, C. Gao, C. Huang, C. Lv, C. Zheng, D. Liu, F. Zhou, F. Huang, F. Hu, H. Ge, H. Wei, H. Lin, J. Tang, J. Yang, J. Tu, J. Zhang, J. Yang, J. Yang, J. Zhou, J. Zhou, J. Lin, K. Dang, K. Bao, K. Yang, L. Yu, L. Deng, M. Li, M. Xue, M. Li, P. Zhang, P. Wang, Q. Zhu, R. Men, R. Gao, S. Liu, S. Luo, T. Li, T. Tang, W. Yin, X. Ren, X. Wang, X. Zhang, X. Ren, Y. Fan, Y. Su, Y. Zhang, Y. Zhang, Y. Wan, Y. Liu, Z. Wang, Z. Cui, Z. Zhang, Z. Zhou, and Z. Qiu, “Qwen3 technical report,” 2025.
- [13] D. M. Adhikari, A. Hartland, I. Weber, and V. K. Cannanure, “Exploring LLMs for automated generation and adaptation of questionnaires,” in Proceedings of the 7th ACM Conference on Conversational User Interfaces (CUI ’25), 2025. arXiv:2501.05985.
- [14] A. Fuchs, A.-C. Haensch, and W. Weber, “AI for survey design: Generating and evaluating survey questions with large language models.” SocArXiv preprint, 2026.
- [15] GESIS – Leibniz Institute for the Social Sciences, “SQP 3.0: Survey quality predictor.” <https://sqp.gesis.org/>, 2022.